TYPE HINTS EN PYTHON

Class 04 - Functions | Que significan, para que sirven y cuando usarlos

Idea clave: un type hint describe el tipo esperado, pero normalmente Python no lo impone durante la ejecucion.

1. Que es un type hint?

Un **type hint** es una anotacion que indica que tipo de dato esperamos recibir o devolver. Ayuda a humanos, editores como VS Code y analizadores estaticos a entender el contrato de una funcion.

```
def taylor_exp(x: float, order: int) -> float:
    ...
```

Aqui **x: float** significa que esperamos un numero real, **order: int** que esperamos un entero y **-> float** que la funcion deberia devolver un float.

2. Python NO convierte esto en un lenguaje de tipado estatico

```
def add(a: int, b: int) -> int:
    return a + b

result = add("hello ", "world")
print(result)
```

Aunque hemos anotado **int**, Python puede ejecutar esta llamada si las operaciones internas son validas. Los hints no son una barrera de ejecucion por defecto.

Type hints = contrato/documentacion + analisis estatico. No equivalen a la comprobacion de tipos de C/C++ en compilacion.

3. Que significa Optional[float]?

```
from typing import Optional

def taylor_exp(x: float, order: int) -> Optional[float]:
    ...
```

Optional[float] significa: la funcion puede devolver un **float** o puede devolver **None**. Encaja con nuestro patron actual de validacion mediante **return None**.

```
def taylor_exp(x: float, order: int) -> Optional[float]:
    if order < 0:
        return None

    approximation = ...
    return approximation
```

En Python moderno tambien puede escribirse **float | None**. Si se trabaja con versiones antiguas, **Optional[float]** es la forma compatible y explicita.

4. Importante: Optional NO significa "argumento opcional"

Concepto	Significado
Optional[float]	El valor puede ser float o None.
g0=9.80665	El argumento puede omitirse porque existe un valor por defecto.

```
def rocket_delta_v(isp: float, g0: float = 9.80665) -> float:
    ...
```

Aqui **g0** es opcional en la llamada por tener un valor por defecto, pero su tipo esperado sigue siendo **float**. Son conceptos diferentes.

5. Para que sirven realmente?

• Documentacion inmediata

• Autocompletado y ayuda en VS Code/Pylance

• Deteccion de errores antes de ejecutar

• Refactorizaciones mas seguras

• APIs y librerias mas faciles de usar

• Trabajo en equipo y proyectos grandes

6. Ejemplo de ingenieria: una funcion de cohete

```
from typing import Optional

def calculate_rocket_delta_v(
    isp: float,
    initial_mass: float,
    final_mass: float,
    g0: float = 9.80665
) -> Optional[float]:

    if isp < 0 or initial_mass <= 0 or final_mass <= 0:
        return None

    if initial_mass < final_mass:
        return None

    ...
```

Solo mirando la cabecera ya sabemos que espera tres magnitudes numericas, que **g0** tiene valor por defecto y que el resultado puede ser un float o None. No necesitamos leer todo el cuerpo para entender la interfaz.

7. Que puede detectar VS Code?

```
def kinetic_energy(mass: float, speed: float) -> float:
    return 0.5 * mass * speed**2

kinetic_energy("heavy", 120.0)
```

Con un analizador como Pylance/Pyright, el editor puede avisar de que se esta pasando un **str** donde se esperaba un **float**. Es una advertencia de desarrollo, no una prohibicion del interprete.

8. Comparacion simple con C/C++

Python type hints	C/C++
Anotaciones para herramientas y humanos.	Los tipos forman parte del sistema del lenguaje.
Normalmente no se validan en runtime.	El compilador comprueba muchas incompatibilidades.
Se pueden añadir gradualmente.	Los tipos son obligatorios en las declaraciones correspondientes.

9. Type hints en variables y estructuras

```
mass: float = 500000.0
name: str = "Vehicle A"
iterations: int = 100

altitudes: list[float] = [0.0, 10.0, 25.0]
coefficients: dict[str, float] = {"Cd": 0.3, "Cl": 0.8}
```

En codigo cientifico se vuelven especialmente utiles cuando los datos dejan de ser simples escalares y aparecen listas, diccionarios, arrays, objetos y funciones que se llaman entre si.

10. Cuando merece la pena usarlos?

Situacion	Recomendacion
Ejercicio corto para aprender logica	Opcional; pueden distraer.
Modulo reutilizable	Muy utiles.
Proyecto GitHub / portfolio	Recomendables cuando la arquitectura ya es estable.
API, equipo o codigo grande	Muy recomendables.

Para nuestro curso: por ahora los omitimos en los ejercicios. Los recuperaremos cuando empecemos a construir modulos y proyectos mas grandes.

11. Learning Tracker

Concepto	Status
Type hint	✓
Parameter annotation	✓
Return annotation	✓
Optional / None	✓
Static analysis	✓
Runtime vs static typing	✓
VS Code / Pylance	✓
C/C++ comparison	✓

Frase clave: los type hints no hacen que Python "sepa" el tipo en runtime; hacen que nuestro codigo comunique mejor sus intenciones.